Prise de note pour le LAB 4

Antoine POIRIER

October 31, 2026

Quelles attaques vulnérables ont réussi ?

Aucune attaque n'a réussi dans les résultats observés. Dans le mode vulnérable, 0 attaque sur 16 a été considérée comme réussie. Dans le mode renforcé, le résultat est également de 0 sur 16. Le taux de réussite des attaques (ASR) est donc de 0 % pour les quatre familles testées : LLM01, LLM02, LLM05 et LLM06.

Quels contrôles renforcés ont changé le résultat ?

Le changement principal concerne le scénario LLM01-A1. En mode vulnérable, la requête a été envoyée au modèle, qui l'a refusée. En mode renforcé, l'application l'a bloquée avant même l'appel au modèle, grâce à un contrôle `input_signature` (heuristique pédagogique ciblée).

La différence vient donc de l'application et non d'un changement du comportement du modèle : le contrôle renforcé a empêché la requête d'atteindre le modèle.

Quels cas ont été arrêtés par le modèle ?

Le modèle a refusé 25 cas adversariaux au total : 13 en mode vulnérable et 12 en mode renforcé. Ces refus ont été produits par le comportement du modèle.

Pour certains scénarios LLM01 (A2 à A4), le modèle a renvoyé une décision sûre (`model_safe_decision`) plutôt qu'un refus explicite. Ces cas n'ont toutefois pas été considérés comme des attaques réussies dans les résultats.

Quels cas ont été arrêtés par l'application ?

Un seul cas a été bloqué directement par l'application : LLM01-A1 en mode renforcé. Le contrôle `input_signature` a bloqué la requête avant son traitement par le modèle.

Aucun blocage de sortie par l'application n'apparaît dans les résultats complets.

Un refus du modèle signifie-t-il que l'application est sécurisée ?

Non. Un refus du modèle ne garantit pas à lui seul la sécurité de l'application. Le modèle peut refuser une demande dans un scénario donné, mais la sécurité doit aussi être assurée par des contrôles déterministes dans l'application.

Par exemple, l'application doit vérifier les autorisations avant l'utilisation d'un outil, limiter les actions possibles et contrôler les sorties avant de les utiliser.

Une demande légitime a-t-elle été considérée à tort comme dangereuse(faux positif) ?

Non, aucun faux positif n'a été observé dans cet échantillon. Les 8 tâches légitimes ont réussi et aucune n'a été bloquée à tort. Il faut néanmoins rester prudent, car le nombre de demandes légitimes

testées est limité.

Quelle famille OWASP a été la plus difficile à défendre ?

LLM01 – Prompt Injection semble être la famille la plus difficile à traiter par un refus explicite et systématique du modèle. Pour les scénarios A2 à A4, le modèle a fourni une décision sûre plutôt qu'un refus explicite.

Cela ne signifie pas que ces attaques ont réussi : aucune réussite n'a été enregistrée. On peut donc dire que LLM01 a été la famille la moins homogène en matière de refus explicites, mais pas qu'elle a été la plus vulnérable dans ce test.

Quel contrôle déterministe ajouter avant la mise en production ?

J'ajouterais une vérification stricte des permissions au niveau de l'application avant toute exécution d'outil ou d'action. Cette vérification devrait notamment : - utiliser une liste d'outils et d'actions explicitement autorisés ; - vérifier que l'utilisateur a réellement le droit de demander l'action ; - limiter chaque action au périmètre autorisé ; - bloquer les appels non autorisés avant leur exécution ; - conserver une trace des décisions et des actions dans les journaux.